

Praktikum Strojnega Učenja

VAE in nova fizika

Januar 2024

1 Nova fizika na LHC

Po več desetletjih intenzivnih raziskav se je zgodba Standardnega Modela (SM) fizike osnovnih delcev leta 2012 z odkritjem Higgsovega bozona lepo zaokrožila. S tem so bile odkrite vse napovedane prostostne stopnje. Raziskave seveda še potekajo, saj vseh lastnosti SM še nismo postavili na preizkušnjo, kljub temu pa se SM različnim testom znova in znova zoperstavlja [1]. Nekaj ključnih lastnosti pa mu vseeno manjka, najbolj očitne so nevtrinske mase, za katere iz meritve nevtrinskih oscilacij vemo, da so neničelne, v nasprotju z napovedjo SM. Povedano drugače, ne razumemo mehanizma, preko katerega nevtrini dobijo svojo (majhno) maso, v nasprotju z ostalimi fermioni SM, pri katerih smo vedno bolj prepričani v Higgsov mehanizem. Poleg tega je zelo očiten problem tudi temna snov, ki v SM nima kandidata. Med konceptualnimi problemi se znajdejo problem hierarhije, problem okusa, problem močne simetrije CP, in drugi [2].

Kljub velikemu številu indikatorjev po potrebi fizike onkraj SM (nove fizike) trenutno ni jasno, kako model razširiti. Idej je veliko, od supersimetrije do dodatnih dimenzij in kompozitnih modelov. Ob vselej večjem številu kandidatov modelov in pomanjkanju odkritij novih prostostnih stopenj ali jasnih indikatorjev odstopanj od napovedi SM na trkalnikih kot je LHC se pojavi vprašanje, kako sploh iskati novo fiziko? Ob vseh modelih, ki so na voljo, je težko izbrati tistega, ki mu bomo posvetili več pozornosti, kot ostalim. Zato je treba najti načine iskanja, ki so univerzalni, oziroma neodvisni od modela. Izmed mnogih je takšna recimo metoda *bump hunt*, ki išče resonančne strukture in je bila med drugim uporabljena za iskanje Higgsovega bozona. Težava v tej metodi je ta, da poleg resonančnega vrha ni preveč senzitivna na ostale potencialno pomembne lastnosti signala.

Metode strojnega učenja kažejo velik potencial za uporabo prav v iskanju nove fizike v kontekstu visoko-energijskih trkalnikov. Ker ne vemo, kakšen je signal v resnici, se naravno ponujajo metode nenadzorovanega učenja. Le-te so dandanes predmet intenzivnih raziskav (npr. [3]). Primer, ki si ga bomo ogledali tukaj, temelji na LHC Olimpijadi (LHCO) [4]. Iskali bomo dvocurkovni signal v morju dogodkov ozadja, tako da bomo ustvarili klasifikator, ki bo deloval na podlagi latentnega prostora Variacijskega AvtoEnkoderja (VAE) [5]. Poleg klasifikatorja VAE se lahko v določenih primer uporabi za generacijo novih podatkov.

Naloge so razdeljene na 3 dele. Prvi del se ukvarja s klasifikacijo in generacijo števk iz MNIST podatkov z uporabo binarne mešanice, v drugem delu se uporabi VAE za klasifikacijo in generacijo števk iz MNIST in v tretjem delu se uporabi VAE na podatkih LHCO z namenom odkritja dvocurkovnega signala.

2 Binarna mešanica in MNIST

Podatkovno zbirko MNIST sestavljajo slike števk velikost 28×28 pixels. Vsak pixel lahko zavzame celoštevilsko vrednost med 0 in 255. Podatkovno lahko enostavno naložimo z recimo ukazom:

```
from sklearn.datasets import fetch_openml
X, y = fetch_openml("mnist_784", version=1, return_X_y=True, as_frame=False)
x = X/255
y = np.array(y, dtype = int)
```

Tako je i -ta slika predstavljena z nizom $x[i]$ velikosti $28 \cdot 28$. Vrednosti komponent vsakega niza so med 0 in 1. $y[i]$ pa predstavlja številko slike, zavzame vrednosti in množice $\{0, 1, 2, \dots, 9\}$.

Točke (slike) v $28 \cdot 28$ dimenzionalnem prostoru lahko pogručimo z uporabo binarne mešanice. Predpostavimo 10 gruč. Cilj je, da vsako gručo opišemo s produktom Bernoulijevih porazdelitev:

$$P(x|\mu) = \prod_{i=1}^N \mu_i^{x_i} (1 - \mu_i)^{(1-x_i)}, \quad (1)$$

z neznanimi parametri $\mu = (\mu_1, \dots, \mu_{28 \cdot 28})$. Celotna porazdelitev v tem prostoru je utežena vsota porazdelitve posamezne gruče:

$$\mathcal{P}(x) \equiv P(x|\mu^1, \dots, \mu^{10}, \pi_1, \dots, \pi_{10}) = \sum_{j=1}^{10} P(x|\mu^j) \pi_j, \quad (2)$$

kjer so π_j za $j = 1, \dots, 10$ uteži, ki zadoščajo naslednjim lastnostim: $\pi_j \in [0, 1]$ za vsak j in $\sum_j \pi_j = 1$. Uteži π in parametre μ_k^j , kjer $k = 1, \dots, 28 \cdot 28$, v splošnem dobimo tako, da maksimiziramo log-likelihood:

$$\sum_x \log(\mathcal{P}(x)) = \max. \quad (3)$$

V metodi binarnih mešanic je maksimizacija likelihooda izvedena posredno z naslednjim algoritmom:

1. Izberi začetne vrednosti parametrov μ_k^j (lahko jih naključno izberemo z uporabo enakomerne porazdelitve na intervalu $[0, 1]$). Izberi vrednosti za π_j (recimo: naključna izbira iz Dirichletove porazdelitve s parametrom $\alpha = (1, \dots, 1)$.)
2. Ponavljaj naslednja koraka dokler se parametri ne signifikantno spremenijo.

E: Izračunaj verjetnost $P(x|k)$, da slika x_l pripada gruči j :

$$P(x_l|j) = \frac{P(x_l|\mu^j)}{\mathcal{P}(x_l)}, \quad (4)$$

to stori za vsako sliko in za vsako gručo (Na predavanjih je $P(x_l|j)$ označen z γ).

M: Posodobi vrednosti parametrov z uporabo naslednjih formul:

$$\mu^j = \frac{\sum_x P(x|j)x}{\sum_{j=1}^{10} \sum_x P(x|j)}, \quad (5)$$

$$\pi_j = \frac{\sum_x P(x|j)}{\sum_{j=1}^{10} \sum_x P(x|j)}. \quad (6)$$

2.1 Naloge

1. Implementiraj algoritem binarnih mešanic in ga preizkusi na podatkih MNIST. Nariši slike števil, ki so predstavljene z nizi μ^j , primer kode:

```
plt.figure()
plt.imshow(mu[j].reshape((28, 28)))
plt.show()
```

Nato poskusi generirati nekaj novih slik tako, da si izbereš (dovolj 1) gručo j in naključno izžrebaš nekaj slik iz porazdelitve $P(x|\mu^j)$, primer kode:

```
x_new = np.random.binomial(n=1, p = mu[j]).
```

V zgornji kodi za $n = 1$ imamo Bernoullijevo porazdelitev. Z večanjem parametra n so generirane slike lepše. Ali bi bilo smiselno uporabiti iz vsega začetka binomsko porazdelitev namesto Bernoullijeve? Pri implementaciji morda pride prav trivialen trik:

$$\frac{e^{A_j}}{\sum_j e^{A_j}} = \frac{e^{A_j+B}}{\sum_j e^{A_j+B}}, \quad (7)$$

ki velja za vsak B .

3 VAE in MNIST

V tem delu uporabimo VAE na podatkih MNIST, katere lahko naložimo z ukazom iz prejšnjega poglavja ali recimo na naslednji način:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import numpy as np

(x_train, y_train), _ = mnist.load_data()

x_train = x_train.astype('float32') / 255.0

x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
```

`x_train` vsebuje 60000 nizov velikosti $28 \cdot 28 = 784$. Vrednosti so normalizirani med 0 in 1 kar običajno pripomore k stabilnejšem učenju VAE. Implementacija VAE je enostavna, na spletu obstaja veliko literature in implementacij (GitHub) v različnih programskih jezikih in knjižnicah. Spodaj je podana kratka razlaga o implementaciji VAE v `keras`-u (predstavitev nalog vsebuje tudi kratko razlago implementacije).

Enkoder je običajna mreža, sestavljena iz nekaj slojev in aktivacijskih funkcijah. Enkoder prejme `x_train` in vrne `z_mean` in `z_log_var`, ki sta enakih dimenzij kot vhodni "tenzor". Primer implementacije z eno skrito linearno plastjo (Pytorch: linear layer, Tensorflow: dense layer) (v primeru, da so vhodni podatki slike torej, da je `x_train` dimenzije $6000 \times 28 \times 28$ bi uporabili 2D konvolucijske plasti):

```
original_dim = np.prod(x_train.shape[1:]) # dimenzija vhodnih podatkov
hidden_dim = 64 # skriti sloj z 64 node-i
latent_dim = 2 # 2D latentni prostor

inputs = keras.Input(shape=(original_dim,))
h = keras.layers.Dense(hidden_dim, activation='selu')(inputs)
z_mean = keras.layers.Dense(latent_dim)(h)
z_log_var = keras.layers.Dense(latent_dim)(h)
```

Vzorčenje z je izvedeno preko reparametrizacije:

$$z = z_{\text{mean}} + \exp\left(\frac{1}{2}z_{\text{logvar}}\right)\epsilon,$$

kjer je ϵ naključna spremenljivka normalne porazdelitve $\mathcal{N}(0, 1)$. Primer implementacije:

```
def sampling(args):
    z_mean, z_log_var = args
    epsilon = K.random_normal(shape=(K.shape(z_mean)[0], latent_dim), mean=0.0, stddev=1.0)
    return z_mean + K.exp(0.5*z_log_var)*epsilon
```

Pri tem je treba definirati poseben sloj (obstajajo tudi druge rešitve), ki deluje na vhodne podatke z zgornjo funkcijo:

```
z = keras.layers.Lambda(sampling)([z_mean, z_log_var])
```

S spodnjim ukazom je Enkoder implementiran in zajet v objektu `encoder`:

```
encoder = keras.Model(inputs, [z_mean, z_log_var, z], name='encoder')
```

Implementacija dekoderja je enostavnejša. Ta prejme z in vrne rekonstrukcijo nizov x . Spodaj je primer implementacije dekoderja z 1 skrito plastjo.

```
latent_inputs = keras.Input(shape=(latent_dim, ), name='z_sampling')
x = keras.layers.Dense(hidden_dim, activation='selu')(latent_inputs)
outputs = keras.layers.Dense(original_dim, activation='sigmoid')(x)
decoder = keras.Model(latent_inputs, outputs, name='decoder')
```

Zadnja aktivacijska funkcija dekoderja je `sigmoid` funkcija, katere zaloga vrednosti je interval $[0,1]$. Enkoder in dekodeer povežmo skupaj z naslednjimi ukazami:

```
outputs = decoder(encoder(inputs)[2])
vae = keras.Model(inputs, outputs, name='vae')
```

S tem so vse komponente (razen cenovne funkcije in optimizerja) VAE definirane. V zgornji kodi, prva vrstica: dekodeer prejme 3. izhod enkoderja, to je z . Najbolj primeren optimizator je za večino primerov `adam`. Cenovna funkcija za VAE je bila izpeljana na predavanjih. Sestavljena je iz dveh delov: cenovne funkcije, ki primerja vhodne podatke z izhodne podatke, ter in KL divergence. VAE je s spodnjimi ukazi v celoti definiran.

```
rec_loss = keras.losses.binary_crossentropy(inputs, outputs)
rec_loss *= original_dim
kl_loss = -0.5*K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var), axis=-1)
vae_loss = K.mean(rec_loss + kl_loss)
vae.add_loss(vae_loss)
vae.compile(optimizer='adam')
```

Faktor med obema deloma cenovne funkcije 784 (`original_dim`) je arbitraren. Z večanjem tega se rekonstrukcija izhodnih spremenljivk izboljša, vendar so tipično točke v latentnem prostoru bolj oddaljene med seboj. VAE se začne učiti, ko poženemo ukaz:

```
batch_size = 32
vae.fit(x_train, x_train, epochs=100, batch_size=batch_size, validation_data=None)
```

Pri učenju tipično ne pride do overfittanja, če je dimenzija latentnega prostora dovolj majhna. Zato testnih in validacijskih podatkov (v tem primeru) ne potrebujemo. Takoj po učenju shranimo parametre naučenega VAE z ukazom:

```
np.save("weights.npy", vae.get_weights())
```

S tem ni potrebno vsakič znova učiti VAE. Shranjene parametre naložimo preko

```
vae.set_weights(np.load("weights.npy", allow_pickle = True))
```

Bolj napredne metode shranjevanja parametrov modela in stanje optimizerja (zgornja koda ne shrani stanje optimizerja) so opisane v dokumentaciji `keras/tensorflow`.

Naučen VAE lahko uporabimo za slikanje vhodnih podatkov v latentni prostor in tam recimo izvedemo gručenje (to ni del te naloge, v praksi se uporablja modificirane VAE, recimo GMMVAE, VADE,... Taki modeli se naučijo slikati in gručiti podatke v latentnem prostoru). Do enkoderja lahko dostopimo na naslednji način (enkoder vrne `z_mean`, `z_log_var`, `z`):

```
z = encoder.predict(x_train)[2]
```

Podobno za dekode:

```
x_reconstructed = decoder.predict(z)
x_image = x_reconstructed.reshape(28, 28)
# x_images = x_reconstructed.reshape(-1, 28, 28)
```

3.1 Naloge

2. Implementiraj VAE po navodilih in ga zaženi na podatkih MNIST z `adam` optimizerjem do epocha 100.
3. Nariši točke v 2D latentnem prostoru tako, da vsaki številki pripada druga barva (primer grafa je prikazan v predstavitvi nalog).
4. Generiraj slike iz vsake točke na smiselno izbrani 2D mreži, ki dobro pokriva točke v latentnem prostoru (primer rešitve je prikazan v predstavitvi nalog).

4 VAE in mase neznanih delcev

V nadaljevanju uporabimo VAE za razločevanje dogodkov signala od dogodkov ozadja. VAE slika podobne vhodne podatke blizu skupaj v latentnem prostoru, torej je smiselno pričakovati, da bodo dogodki signala v latentnem prostoru lokalizirani. Tipično je število dogodkov signala veliko manjše kot dogodkov ozadja, kar pomeni, da bo VAE boljše rekonstruiral dogodke iz ozadja. Vemo še, da če dogodki ozadja in signala pripadajo dovolj različnim porazdelitvam, je smiselno tudi pričakovati, da bodo dogodki signala v latentnem prostoru lokalizirani na robovih porazdelitve dogodkov ozadja. Če obstaja klasifikator v latentnem prostoru za katerega je znano, kje se pojavijo dogodki signala, ga lahko uporabimo za izluščanje lastnosti signala ter generacijo novih dogodkov signala.

V nadaljevanju se bomo ukvarjali s procesom trka dveh protonov, ki ustvarita fiktiven delec Z' . Ta razpade na dva neznana produkta X in Y . Oba nato razpadeta na znane delce ter s tem ustvarita curek delcev. Oba curka detektiramo. V primeru, ko protona ne ustvarita delec Z' detektiramo ozadje. Na tak način imamo dogodke signala (protona ustvarita Z) in dogodke ozadja.

Uporabili bomo 2 tipa podatkov, testni podatki in podatki črne škatle. Pri testnih podatkih poznamo mase delcev X , Y , Z' in kateri dogodki so signal in ozadje, medtem ko pri podatkih črne škatle ne poznamo mas in ne vemo kateri dogodki predstavljajo signal in ozadje. Cilj je pripraviti metodo za določanje mas delcev X , Y in Z' z uporabo VAE na testnih podatkih in nato jo preizkusiti na podatkih črne škatle (zadnje dve nalogi).

4.1 Podatki

Testni podatki so pripravljeni za primera, ko je razmerje med številom dogodkov signala in ozadja $S/B = 10\%$ (H) in, ko je to razmerje $S/B = 0.1\%$ (L). Za vsak primer imamo 3 datoteke:

- `lhco_H_jetobs.npy` Vsebuje $\sim 10^6$ dogodkov, vsak je opisan z 8 spremenljivkami. VAE treniramo s temi podatki (H je za $S/B = 10\%$).
- `lhco_H_labels.npy` Oznake dogodkov. Če je oznaka 0 je slednji dogodek ozadje, če je 1 pa signal. Te podatke uporabimo samo za risanje grafov in izračun AUC in ROC krivulj.
- `lhco_H_invmass.npy` Za vsak dogodek je podana invariantna masa. VAE ne treniramo s temi podatki.

Podatki črne škatle vsebujejo dve datoteki:

- `blackbox_jetobs.npy`,
- `blackbox_invmass.npy`.

Oznake dogodkov niso znane.

Datoteke, ki se končajo z `*jetobs.npy` vsebujejo približno 10^6 dogodkov, vsak je predstavljen z 8 spremenljivkami v tem vrstnem redu: m_{j_1} , $(\tau_2/\tau_1)_1$, $(\tau_3/\tau_2)_1$, $m_{d_{j_1}}$, m_{j_2} , $(\tau_2/\tau_1)_2$, $(\tau_3/\tau_2)_2$, $m_{d_{j_2}}$. Prve štiri so za prvi hadronski curek, ostale štiri pa za drugi hadronski curek. m_{j_1} in m_{j_2} sta masi hadronskega curka, ki imata največjo transversalno gibalno količino od vseh detektiranih curkov (podobno kot pri 6. nalogi za PSUF). Količini $(\tau_2/\tau_1)_1$, $(\tau_3/\tau_2)_1$ se imenujeta *N-subjettiness* [6] podajata implicitno verjetnost, da je (v tem primeru prvi) hadronski curek nastal iz dveh ali treh osnovnih delcev. Količina $m_{d_{j_1}}$ je vsota *massdrop*-ov. *massdrop* je največje razmerje mas gruč na nivoju j in gruč na nivoju $j + 1$ pri hierarhičnem gručenju (za več informacij glej 6. nalogo za PSUF).

VAE treniramo na transformiranih podatkih `*jetobs.npy`, in sicer podatke transformirano na naslednji način:

```
from sklearn.preprocessing import QuantileTransformer

x_train = np.load(x_path)
x_train = x_train.astype('float32')
f_scaler = QuantileTransformer(output_distribution='uniform')
x_train_transformed = f_scaler.fit_transform(x_train)
```

kjer transformacija `QuantileTransformer` preslika podatke tako, da je postanejo porazdelitve spremenljivk enakomerne. Izkazuje se, da ta transformacija izboljša učenje VAE. V praksi je seveda treba poskusiti različne transformacije podatkov. VAE se torej uči na podatkih `x_train_transformed`, kar pomeni, da dekodek poskuša rekonstruirati transformirane podatke, ki jih recimo imenujemo `x_train_reconstructed_transformed`. Te, če je potrebno, transformiramo nazaj z ukazom:

```
x_train_reconstructed = f_scaler.inverse_transform(x_train_reconstructed_transformed)
```

Inverzna transformacija pride prav pri risanju določenih grafov, med učenjem VAE se jo ne uporablja.

4.2 Naloga: LHCO

Za reševanje tega dela naloge lahko uporabiš kodo iz dela naloge o MNIST, z nekaj modifikacijami. Tako v `enkoder`-ju, kot v `dekoder`-ju sedaj uporabimo 3 skrite sloje z 64 node-i, namesto enega (še vedno pa na vsakem uporabimo aktivacijsko funkcijo `selu`). V izhodnem sloju `dekoder`-ja sedaj ne uporabimo aktivacijske funkcije `sigmoid`, ampak uporabimo kar linearno. V definiciji sloja `outputs` torej lahko kar izbrišemo `activation='sigmoid'`. Uporabili bomo tudi večji `batch_size=1000` in samo eno latentno dimenzijo, `latent_dim=1` (torej bomo dobili samo en z_{mean} in z_{logvar} na dogodek). Namesto binarne navzkrižne entropije (ang. binary cross entropy) za rekonstrukcijsko cenovno funkcijo sedaj uporabimo raje `rec_loss = keras.losses.mean_squared_error(inputs, outputs)`, njen multiplikator pa postavimo na 5000, torej namesto `rec_loss *= original_dim` uporabimo `rec_loss *= 5000`.

4.2.1 Naloge, ki se nanašajo na testne podatke

5. Nariši porazdelitve prvih 4 spremenljivk za signal, ozadje in oboje skupaj (4 grafi) za podatke z $S/B = 10\%$. Nato transformiraj te iste podatke z `QuantileTransformer` in tako kot prej nariši njihove porazdelitve (spet 4 grafi). Preveri še, da inverzna transformacija deluje.
6. Implementiraj VAE z ustreznimi hiperparametri, glej poglavje 4.2, ter ga natreniraj z uporabo optimizatorja `adadelta` na podatkih z $S/B = 10\%$ in $S/B = 0.1\%$ do epocha 100. Postopek ponovi vsaj 3-krat in shrani uteži modela.
7. Z enkoderjem naučenega VAE preslikaj vse dogodke v latentni prostor. Nariši porazdelitve signala in ozadja za spremenljivke z_{mean} , z_{logvar} in z_{mean}^2 za podatke z $S/B = 10\%$ in $S/B = 0.1\%$ (2 grafa za vsak naučen VAE). Kje se nahaja večina dogodkov signala? Pojasni ugotovitve (ta del je ključen pri reševanju nalog, ki uporabljajo podatke črne škatle).
8. Nariši ROC krivuljo za klasifikatorja z_{mean}^2 in z_{logvar} za $S/B = 10\%$ in $S/B = 0.1\%$ ter izračunaj AUC. *Namig: S knjižnico `sklearn` se lahko uporabijo naslednji ukazi:*

```
from sklearn import metrics
auc = metrics.roc_auc_score(labels, measure)
fpr, tpr, thresholds = metrics.roc_curve(labels, measure)
```

kjer je *measure* izbran klasifikator $\pm z_{\text{mean}}^2$ in $\pm z_{\text{logvar}}$ (predznak določa iz katere smeri začnemo rezati podatke, izberi ga tako, da bo AUC največji) Nazadnje nariši graf funkcije $1/fpr$ (po ang. inverse false positive rate) v odvisnosti od *tpr* (ang. true positive rate).

9. Razvrsti netransformirane podatke za invariantno maso, m_{j1} in m_{j2} po naraščajoči ali padajoči vrednosti klasifikatorja z največjim AUC (glej prejšnjo točko) tako, da bo večina dogodkov signala na začetku seznama. Vzami prvih 500, 1000 in 2000 dogodkov iz tega seznama in poskusi določiti maso neznanega delca in masi obeh produktov z negotovostjo *Iz porazdelitev m_{j1} se lahko oceni masa in negotovost prvega curka, morda se splača omejiti dogodke na invariantne mase med 3.3 TeV in 3.7 TeV.*
10. Izberi 1 naučen VAE in preveri kako dobro rekonstruira porazdelitve prvih 4 spremenljivk (ne pozabi na inverzno transformacijo).
11. **Dodatna:** Treniraj še en VAE tako, da med učenjem izračunaš AUC ob koncu vsakega epoch-a. Nariši graf funkcije AUC.
12. **Dodatna:** Kako se rekonstrukcija in AUC klasifikatorjev spremeni, če neamesto `adadelta`-e uporabimo `adam` optimizier?
13. **Dodatno:** Kako se rekonstrukcija in AUC klasifikatorjev spremeni, če povečamo dimenzijo latentnega prostora?

4.2.2 Naloge, ki se nanašajo na podatke črne škatle

14. Treniraj VAE na transformiranih podatkih črne škatle. Po učenju izračunaj z_{mean}^2 in z_{logvar} za vsak dogodek in shrani rezultate.
15. Poskušaj določiti maso delca in obeh produktov z ustreznimi negotovostmi tako, da slediš postopku iz 9. točke. Sklepaj kje se nahajajo podatki signala iz ugotovitev pridobljenih iz 7. naloge. *Namig: Morda se bo splačalo omejiti dogodke na invariantne mase med 3.6 TeV in 4.0 TeV.*

Viri

- [1] [link].
URL <https://cds.cern.ch/record/2296980/files/ATL-PHYS-SLIDE-2017-983.pdf>
- [2] J. R. Ellis, Limits of the standard model, in: PSI Zuo2 Summer School on Exploring the Limits of the Standard Model, 2002. [arXiv:hep-ph/0211168](#).
- [3] T. Cheng, J.-F. Arguin, J. Leissner-Martin, J. Pilette, T. Golling, Variational Autoencoders for Anomalous Jet Tagging (7 2020). [arXiv:2007.01850](#).
- [4] [link].
URL <https://lhco2020.github.io/homepage/>
- [5] D. P. Kingma, M. Welling, Auto-encoding variational bayes (2014). [arXiv:1312.6114](#).
- [6] J. Thaler, K. Van Tilburg, Identifying Boosted Objects with N-subjettiness, JHEP 03 (2011) 015. [arXiv:1011.2268](#), [doi:10.1007/JHEP03\(2011\)015](#).